

MODULE 5.1 · Concept 4 of 8

Agents with Memory

Short-term context + long-term vector DB — how state transforms agent capability

9 min

Duration

B3 – Apply

Bloom's

L2 – Intermediate

Level

TH-000003

Prerequisite

AI-AG-FN-TH-000004

Prof. Sashikumaar Ganesan · IISc Bangalore · AhamX Bodhi

SECTION 1 — WHY

Why Memory Changes Everything

A reactive agent forgets. A memory agent remembers, adapts, and improves.

Memory Transforms Agent Capability

Before and after — the same agent with and without memory

Without Memory (Reactive)

- Forgets after each interaction
- Asks user to repeat themselves
- Cannot track progress toward goals
- No personalization — treats every user the same
- Cannot learn from past mistakes
- Each session starts from zero

With Memory (Stateful)

- Remembers past interactions across sessions
- Resolves references ("the late one" → order #4521)
- Tracks what's done vs. what remains
- Adapts tone based on user sentiment history
- Avoids repeating failed approaches
- Builds on previous work

Memory is the single capability that transforms a reactive agent into a useful one.

Short-Term Memory: The Context Window

Fast, bounded, ephemeral — the agent's working memory

What It Holds

- System prompt (instructions, persona, tool descriptions)
- Full conversation history (user messages + agent responses)
- Tool call results from current session
- Working scratchpad (chain-of-thought traces)
- Retrieved context (RAG results injected mid-conversation)

Properties

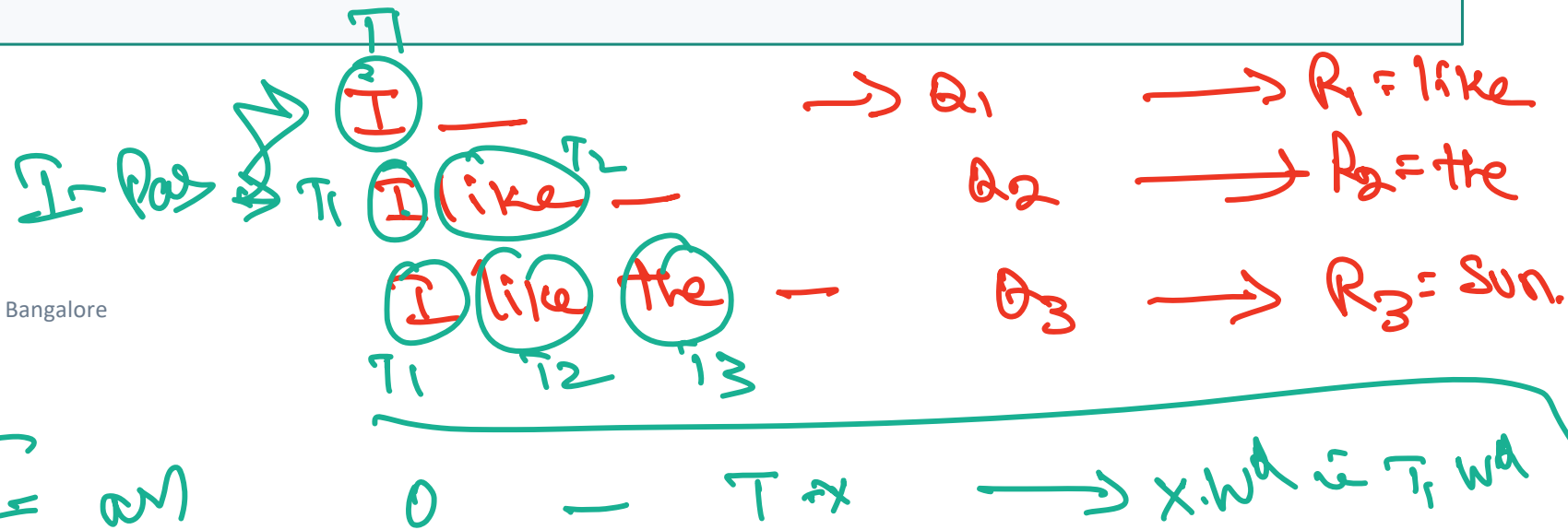
Capacity: 4K tokens (GPT-3.5) to 2M tokens (Gemini 1.5)

Latency: 0 ms — already in the prompt

Persistence: Lost when session ends

Cost: Proportional to token count per call

Analogy: Like RAM — fast but volatile



TP

II Pas

1 > 1 []
Q2

$$x W^K = T_2 W^A$$

$$x W^V = T_3 W^A$$

II Pas $\times \left\{ \begin{matrix} T_1 \\ T_2 \end{matrix} \right\} \xrightarrow{\text{line}} x$

$T_1 W^A, T_2 W^A, T_1 W^K, T_2 W^K, T_1 W^V, T_2 W^V$



Long-Term Memory: External Storage

Persistent, scalable, semantic — the agent's knowledge base

How It Works

1. Agent interaction generates data (conversation, decisions, outcomes)
2. Key information is embedded into vectors (1536-dimensional)
3. Vectors stored in a vector database (Pinecone, Weaviate, etc.)
4. On future queries, agent searches by semantic similarity
5. Top-K relevant memories are injected into the context window
6. Agent reasons with both current context + retrieved memories

Properties

Capacity: Millions of entries — practically unlimited

Latency: ~10–50 ms for retrieval

Persistence: Survives restarts, deployments, crashes

Cost: Storage + embedding + search per query

Analogy: Like a hard disk — slower but permanent

Vector DB

Short-Term vs. Long-Term: Side-by-Side

Six dimensions compared

Dimension	Short-Term (Context)	Long-Term (External)
Capacity	4K–2M tokens	Millions of entries
Latency	0 ms	10–50 ms
Persistence	Session only	Permanent
Retrieval	Full context always present	Semantic similarity search
Cost Model	Per-token per-call	Storage + embed + search
Analogy	RAM	Hard disk + search engine

Production agents use BOTH: context window for immediate state, external memory for everything else.

2,000× Cost Reduction with External Memory

Why you can't stuff everything into the context window

Scenario: Agent needs access to 10,000 past conversation turns

Approach 1: Stuff Everything In-Context

10,000 turns × 100 tokens/turn = 1,000,000 tokens

→ Exceeds GPT-4's 128K window. IMPOSSIBLE.

→ Even if it fit: cost = \$10/query (at \$10/1M input tokens)

Approach 2: Vector Database + Retrieval

Store: 10K vectors × 1536 dims = 60 MB → \$0.33/month on Pinecone

Retrieve: Top-5 relevant turns = 500 tokens injected into context

Cost per query: \$0.005 (500 tokens + embedding + search)

Result: \$10.00 vs \$0.005 = 2,000× cheaper. And it actually works (doesn't exceed window).

SECTION 2 — HOW

Memory in a Customer Support Agent

How memory enables personalized, context-aware support across sessions

Practical: Session 1 – Order Not Arrived

The agent saves context and tracks user sentiment

Turn 1

User: "My order #4521 hasn't arrived"

Agent: "Let me check... shipped 3 days ago, ETA tomorrow."

Memory → Save: *order=4521, status=delayed, sentiment=frustrated*

Turn 2

User: "This always happens with you guys"

Agent: "I understand your frustration. I'll monitor this personally."

Memory → Save: *sentiment=angry, escalation_risk=high*

Turn 3

User: "Fine, I'll wait"

Agent: "I've set a reminder to follow up tomorrow. You'll hear from me."

Memory → Save: *follow_up=scheduled, patience=low*

Practical: Session 2 – Next Day Follow-Up

The agent retrieves context and adapts its tone

Next Day

User: "Hi, any update?"

Agent: "Welcome back! Checking order #4521... delivered 2 hours ago. Can you confirm?"

Memory → Retrieved: order=4521 + sentiment=angry
→ proactive tone

Turn 2

User: "Yes got it, thanks!"

Agent: "Great! I've noted this is resolved. CSAT survey sent. Anything else?"

Memory → Update: resolved=true, CSAT=5/5

Without memory, Session 2 starts cold: 'How can I help you?' With memory: 'Welcome back! Your order #4521 was delivered.'

What Memory Enables: Three Capabilities

Personalization, progress tracking, and cross-session learning

Personalization

Track user preferences, sentiment, communication style. Adapt tone and recommendations. Remember past issues to avoid repeating them.

'Last time you preferred email confirmations. Sending one now.'

Progress Tracking

Know what's been done, what remains, what changed. Resume interrupted tasks. Don't repeat completed steps.

'You completed steps 1–3 yesterday. Continuing from step 4.'

Cross-Session Learning

Remember which approaches worked and which failed. Avoid repeating failed strategies. Build knowledge over time.

'Last time, the refund API timed out. Using the backup endpoint.'

Production Memory Stack

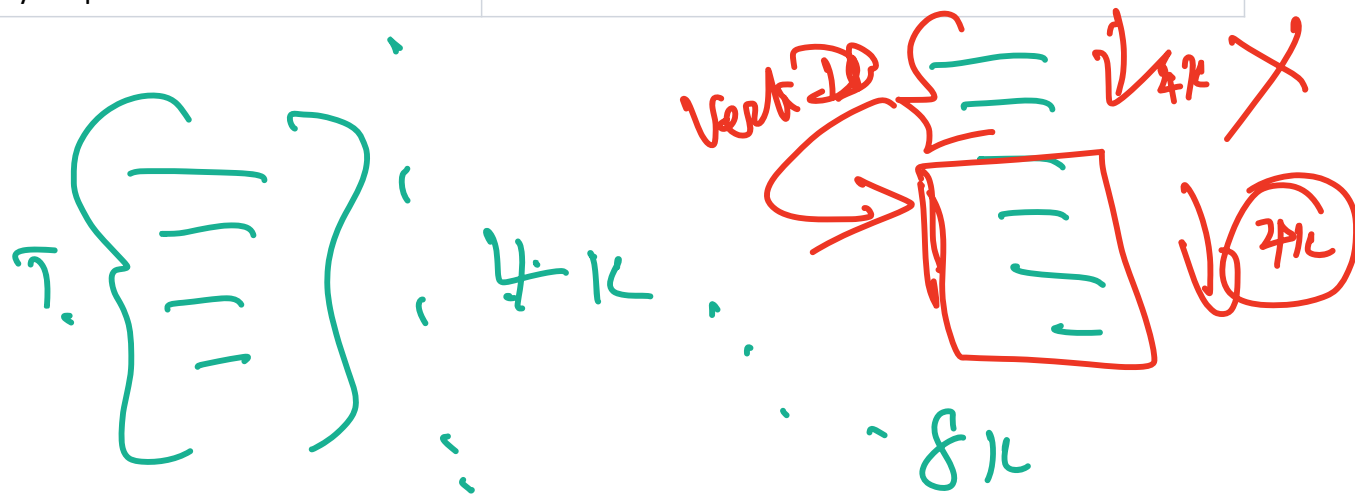
Choosing the right storage technology for each memory type

Technology	Type	Best For	Latency	When to Use
Context Window	Short-term	Immediate conversation	0 ms	Always — every agent has this
Pinecone	Vector DB (cloud)	Semantic past interactions	~20 ms	Large-scale, managed, serverless
Weaviate	Vector DB (hybrid)	Vector + keyword search	~15 ms	Need BM25 + vector combined
Redis	Key-value store	Session state, prefs	~1 ms	Fast lookups, structured data
pgvector	PostgreSQL ext.	Existing SQL infrastructure	~30 ms	Already using PostgreSQL

Memory Failure Modes

What can go wrong — and how to mitigate

Failure Mode	What Happens	Mitigation
Stale Memories	Retrieved memory is outdated. User changed preferences, order was cancelled, policy updated.	Add timestamps. Expire old memories. Prefer recent over old.
Retrieval Misses	Semantic search returns irrelevant results. Query doesn't match stored memory embedding.	Hybrid search (vector + keyword). Query expansion. Reranking.
<u>Context Overflow</u>	Too many retrieved memories exceed the context window. Agent loses track of the goal.	Budget tokens: max 30% for retrieved memory, 70% for current task.
Privacy Leakage	Agent recalls User A's data when talking to User B. Memory not properly scoped.	Namespace memories per user. Access control on retrieval.



Key Takeaways

The essential points from this concept

Two memory types: in-context (fast, bounded, ephemeral) and external (persistent, scalable, slight latency).

External memory is 2,000× cheaper than stuffing everything in-context at scale.

Memory enables: personalization, progress tracking, and cross-session learning.

Production stack: Pinecone/Weaviate for vectors + Redis for KV + context window for immediate state.

Watch for failure modes: stale data, retrieval misses, context overflow, privacy leakage.

Next: AI-AG-FN-TH-000005 — Agent Goals and Objectives

AI-AG-FN-TH-000004 Complete

Agents with Memory

Concept 4 of 8

Advanced Certification in Agentic & Generative AI · IISc Bangalore

Prof. Sashikumaar Ganesan · AhamX Bodhi